
作业四实验报告

王琳睿 241300009 (email: 1637928975@qq.com)

(南京大学 人工智能学院, 南京 210093)

1 三个变量的作用

lr、gamma、buffer_size 这三个变量是深度强化学习中的关键超参数。

1.1 lr -- Learning Rate (学习率) :

作用：控制神经网络参数更新的步长大小, 决定神经网络权重调整的幅度。

在代码中 (agent.py 的 DQNAgent、DDQNAgent 类中) 默认值为 $2e-4$, 传递给神经网络的参数优化器 `self.optimizer = optim.Adam(self.qnetwork_local.parameters(), lr)`, Adam 优化器的更新规则大致为: $\text{参数} = \text{参数} - \text{lr} \times (\text{调整后的梯度})$ 。

lr 取值对深度强化学习的影响:

值越大, 参数更新幅度越大, 学习速度越快, 但可能不稳定, 且可能跳过最优解。

值越小, 参数更新越精细, 学习更稳定, 但收敛速度慢, 且可能陷入局部最优。

1.2 gamma -- Discount Factor (折扣因子) :

作用：平衡当前奖励与未来奖励的重要性。

在代码中 (agent.py 的 DQNAgent、DDQNAgent 类, learn 方法中) 默认值为 **0.99**, 用于计算目标 Q 值: $Q_targets = rewards + (gamma * Q_targets_next * (1 - dones))$, 其非常重视长期的回报, 极力避免游戏结束, 倾向于保持杆子平衡尽可能久, 做出更有远见的决策。

gamma 取值对深度强化学习的影响:

值越接近 1, 智能体越重视长期回报;

值越接近 0, 智能体越重视即时回报。

1.3 buffer_size (经验回放缓冲区大小) :

作用：决定存储多少条经验样本用于训练。

在代码中 (main.py 中), `buffer = deque(maxlen=args.buffer_size)`, 创建了一个双端队列数据结构, 用作经验回放缓冲区, 最大容量即为 buffer_size。默认值 10000, 表示最多存储 10000 条经验样本, 当缓冲区满时, 会自动移除最旧的样本。

buffer_size 取值对深度强化学习的影响:

若 buffer_size 过小, 早期有价值的经验会被挤出, 样本分布偏向近期状态, 即使随机采样, 也无法获得真正的多样性。

当 `buffer_size` 足够大时，能保留各种状态的经验，随机采样能真正打破时间相关性，使得训练更加稳定。缓冲区越大，样本多样性越好，但内存占用越多，`buffer_size` 也不宜过大。

2 两个函数不同之处

顾名思义也可得出，这两个函数的核心区别在于是否进行探索（随机探索而非单纯选择最优）

2.1 函数 `act`：

首先进行神经网络的前向传播，获取当前状态下所有动作的 `Q` 值估计。

然后使用平衡探索和利用的 ϵ -贪婪策略，以 `eps` 概率随机探索，即随机选择一个动作，以 $(1-\text{eps})$ 概率选择最优动作。

该函数更适合在训练阶段使用，用于探索性决策。

2.2 函数 `act_no_explore`：

首先进行神经网络的前向传播，获取当前状态下所有动作的 `Q` 值估计。这部分与 `act` 相同。

然后使用纯贪婪策略，直接返回 `Q` 值最大的动作，不进行随机探索。

该函数更适合在评估时使用，用于获取确定性最优决策。

2.3 训练过程中采用 `act_no_explore` 函数会有什么问题：

智能体会陷入局部最优，永远找不到更好的策略。

网络随机初始化，如果初始策略很差，就会一直执行差的动作，永远没有机会发现更好的动作，则那些状态-动作对的 `Q` 值永远无法更新，导致训练早期就停滞。

得到的数据多样性不足，所有经验都是执行“当前认为最优”的动作，缺少对其他动作结果的了解。

将 `train` 函数中的 `action = agent.act(state, epsilon)` 改为 `action = agent.act(state, eps=0.)`（等价于 `act_no_explore`）：

```
Episode 590 Step 10: Training Loss 0.009263063780963421, Return 10.0
Episode 591 Step 10: Training Loss 0.008117410354316235, Return 9.0
Episode 592 Step 9: Training Loss 0.007431378588080406, Return 10.0
Episode 593 Step 10: Training Loss 0.008252990432083607, Return 10.0
Episode 594 Step 11: Training Loss 0.007402859628200531, Return 9.0
Episode 595 Step 8: Training Loss 0.008005207404494286, Return 10.0
Episode 596 Step 10: Training Loss 0.008198646828532219, Return 10.0
Episode 597 Step 8: Training Loss 0.007796618156135082, Return 9.0
Episode 598 Step 9: Training Loss 0.008558184839785099, Return 10.0
Episode 599 Step 9: Training Loss 0.00746639259159565, Return 10.0
Episode 600 Step 10: Training Loss 0.007577587850391865, Return 8.0
```

如图所示，最终得到的评估策略时的累计回报只在 8.0-10.0 之间徘徊，没有任何增长。训练似乎从起初就已经停滞了。

而修改前得到的：

```
Episode 590 Step 198: Training Loss 0.26416346430778503, Return 215.0
Episode 591 Step 224: Training Loss 0.17772606015205383, Return 210.0
Episode 592 Step 222: Training Loss 0.06576193869113922, Return 235.0
Episode 593 Step 207: Training Loss 0.0657757893204689, Return 210.0
Episode 594 Step 210: Training Loss 0.08389738202095032, Return 194.0
Episode 595 Step 184: Training Loss 0.1731976866722107, Return 225.0
Episode 596 Step 225: Training Loss 0.08279009163379669, Return 244.0
Episode 597 Step 202: Training Loss 0.12091907858848572, Return 208.0
Episode 598 Step 203: Training Loss 0.13083776831626892, Return 246.0
Episode 599 Step 227: Training Loss 0.10611414164304733, Return 206.0
Episode 600 Step 233: Training Loss 0.1382683366537094, Return 226.0
```

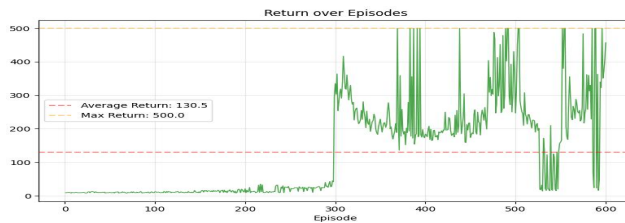
如图所示，最终评估策略时的累计回报可达到 200 以上，且过程中有达到更高值的情况。训练是从低值逐渐增长到高值的。

（一个有意思的现象是，纯贪婪策略的 Training Loss 较小，原因可能是策略一致，Q 值估计自洽，但可能是对当前次优策略的过拟合，因此也可以看出，好的训练应该关注 Return 的增长，而不是单纯追求 Loss 的降低。）

由此可见，训练中采用 `act_no_explore` 最终很难收敛到好的策略。

3 修改强化学习参数

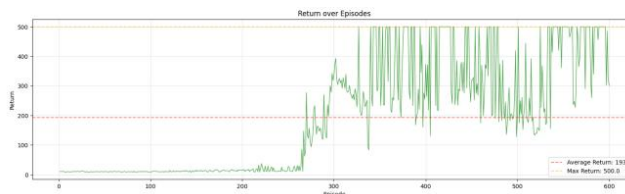
3.1 原参数的训练结果：



平均return: 130.5



平均return: 187.7



平均return: 193.1

（共进行了 3 次，图像分别为以上三张）

大约在 300 episode 左右，return 值会有个激增，在 350~400 episode 左右 return 达到最大值，但后续趋势有所下降，并出现骤升骤降的情况，并非收敛到最优解。

3.2 调参方法：

1、手动调参最常用，可使用网格搜索（若参数较多维数较高也可转用随机搜索）尝试关键配置的经典值组合，但是本情形超参数过多维度过高，这种调参方法效率较低，我决定直接尝试自动化调参。

2、自动优化如贝叶斯优化效率更高，对其进行尝试。

“这是一种基于模型的方法，用于寻找难以评估的目标函数（此处为智能体性能）的最小值或最大值。

其工作原理是：

基于先前评估过的点，建立超参数与性能之间关系的概率替代模型（通常是高斯过程）。

使用采集函数（例如，预期改进、置信上限）来确定下一组要评估的超参数。采集函数平衡了对超参数空间不确定区域的试探性搜寻和对已知表现良好区域的优势发挥。

贝叶斯优化通常比网格搜索或随机搜索更样本高效，尤其是在评估成本高昂时。Optuna、Hyperopt 等工具提供了其实现。”^[3]

部分代码和函数：

```
class BayesianOptimizer: 1个用法 2 Anke *
    def __init__(self, agent_name="dqn", n_trials=50):...

    # 超参数搜索空间
    def objective(self, trial: Trial):...

    # 训练智能体并返回稳定性评估分数
    def train_and_evaluate(self, lr, gamma, epsilon_start, epsilon_end, 1个用法 2 Arike *
                           epsilon_decay_rate, batch_size, buffer_size,
                           update_frequency, lr_decay=0.999, lr_decay_frequency=200,
                           num_episodes=800, max_steps_per_episode=500):...

    # 综合评估性能、稳定性和收敛性
    def comprehensive_evaluation(self, agent, episode_returns, eval_returns, stability_scores):...

    # 评估策略
    def evaluate_policy(self, agent, num_episodes=5):...

    # 执行优化并保存详细结果
    def optimize(self):...

    # 将最佳参数保存为json格式，便于在main.py中调用
    def save_detailed_results(self, study):...
```

其中超参数的搜索空间是这样的，根据查到的资料和大模型的建议决定：

```

lr = trial.suggest_float( name: "lr", low: 1e-5, high: 1e-2, log=True)
gamma = trial.suggest_float( name: "gamma", low: 0.9, high: 0.999)
epsilon_start = trial.suggest_float( name: "epsilon_start", low: 0.8, high: 1.0)
epsilon_end = trial.suggest_float( name: "epsilon_end", low: 0.01, high: 0.1)
epsilon_decay_rate = trial.suggest_float( name: "epsilon_decay_rate", low: 0.98, high: 0.999)
batch_size = trial.suggest_categorical( name: "batch_size", choices: [32, 64, 128, 256])
buffer_size = trial.suggest_categorical( name: "buffer_size", choices: [5000, 10000, 20000])
update_frequency = trial.suggest_categorical( name: "update_frequency", choices: [1, 4, 10, 20])

```

3、随后我了解到还有一些进阶方法，即在训练过程中优化超参数。比如基于种群的训练（PBT）。PBT 是一种结合了进化算法和传统超参数优化的混合方法，专门针对深度强化学习和深度学习的训练过程优化。

“它维护着一个并行训练的智能体群体。定期地：

表现不佳的智能体复制表现较好的智能体的模型权重和超参数（趋利行为）。

复制的超参数随后被随机扰动（试探行为）。

这使得 PBT 能够找到自适应的超参数策略，而非固定值，可能带来更好的最终性能和更快的收敛。它与分布式训练设置结合良好。”

部分代码和函数：

```

class PBTTrainer: 1 个用法 Arike *
    def __init__(self, population_size=10):...

    def should_perform_pbt(self, performances, episode):...

    def exploit_and_explore(self, performances, episode):...

    def get_best_agent(self, performances):...

    def print_population_status(self, performances, episode):...

    def update_performance_history(self, performances, episode):...

    def plot_performance(self, save_path=None):...

    def save_best_parameters(self, best_config, filename=None):...

```

根据贝叶斯优化的结果手动调整（有所缩小）参数范围：

```

# 学习相关参数
'lr': 10 ** np.random.uniform(-5, -3), # 10^-5 到 10^-3
'gamma': np.random.uniform(low: 0.9, high: 0.99),
# 探索相关参数
'epsilon_start': np.random.uniform(low: 0.85, high: 0.95),
'epsilon_end': np.random.uniform(low: 0.05, high: 0.1),
'epsilon_decay_rate': np.random.uniform(low: 0.98, high: 0.999),
# 经验回放参数
'buffer_size': int(np.random.choice([10000, 50000, 100000])),
'batch_size': 2 ** np.random.randint(low: 5, high: 9), # 32 到 512
'update_frequency': int(np.random.choice([1, 4])),

```

在函数 `exploit and explore(self, performances, episode)` 中，多次运行，根据每次运行结果手动调整随机扰动的参数范围，避免扰动过大导致性能骤降或过小导致陷入局部最优，最终为：

```
# 探索策略参数
self.agents[i]['epsilon_start'] = np.clip(
    self.agents[i]['epsilon_start'] * np.random.uniform( low: 0.95, high: 1.05),
    a_min: 0.85, a_max: 0.95
)
self.agents[i]['epsilon_end'] = np.clip(
    self.agents[i]['epsilon_end'] * np.random.uniform( low: 0.9, high: 1.1),
    a_min: 0.05, a_max: 0.1
)
self.agents[i]['epsilon_decay_rate'] = np.clip(
    self.agents[i]['epsilon_decay_rate'] * np.random.uniform( low: 0.995, high: 1.005),
    a_min: 0.98, a_max: 0.999
)
# 学习参数
self.agents[i]['gamma'] = np.clip(
    self.agents[i]['gamma'] * np.random.uniform( low: 0.998, high: 1.002),
    a_min: 0.9, a_max: 0.999
)

self.agents[i]['lr'] = np.clip(
    self.agents[i]['lr'] * np.random.uniform( low: 0.9, high: 1.1),
    a_min: 1e-5, a_max: 1e-4
)
# 经验回放参数
self.agents[i]['buffer_size'] = int(np.clip(
    self.agents[i]['buffer_size'] * np.random.uniform( low: 0.9, high: 1.1),
    a_min: 10000, a_max: 50000
))
self.agents[i]['batch_size'] = int(np.clip(
    self.agents[i]['batch_size'] * np.random.uniform( low: 0.9, high: 1.1),
    a_min: 64, a_max: 512
))
self.agents[i]['update_frequency'] = int(np.clip(
    self.agents[i]['update_frequency'] * np.random.uniform( low: 0.8, high: 1.2),
    a_min: 1, a_max: 10
))
```

3.3 调参结果：

1、贝叶斯优化得到的参数结果（结果注重回报值高，运行 30 轮）：

```
"lr": 1.7034095306011286e-05,
"gamma": 0.9255197625334085,
"epsilon_start": 0.8779905337441186,
"epsilon_end": 0.07051202919814865,
"epsilon_decay_rate": 0.9960959349723897,
"batch_size": 512,
"buffer_size": 5000,
"update_frequency": 1
```

2、贝叶斯优化得到的参数结果（结果注重回报值高且稳定，运行 30 轮）：

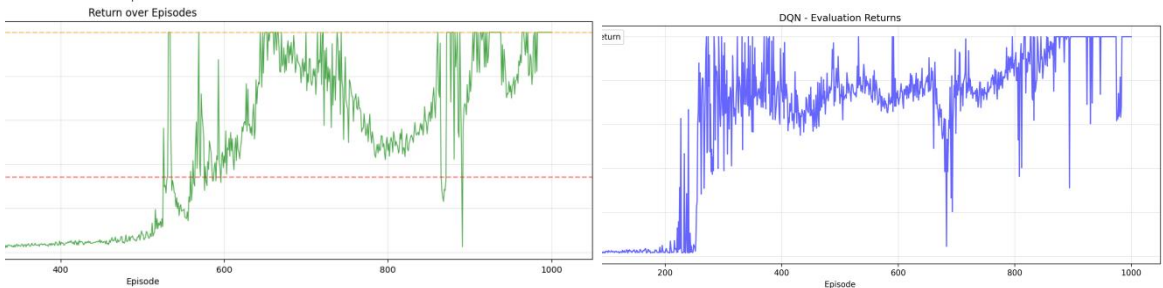

```
"lr": 9.651561560839681e-05,
"gamma": 0.9796907576318281,
"epsilon_start": 0.9195802212881655,
"epsilon_end": 0.09597205990945085,
"epsilon_decay_rate": 0.9885003723080277,
"batch_size": 64,
"buffer_size": 20000,
"update_frequency": 1,
```

3、PBT 得到的参数结果：

```
"lr": 8.858133493954402e-05,
"gamma": 0.9342842593737695,
"epsilon_start": 0.8756324637644137,
"epsilon_end": 0.05306303670081751,
"epsilon_decay_rate": 0.9868255889135402,
"batch_size": 250,
"buffer_size": 37698,
"update_frequency": 1,
```

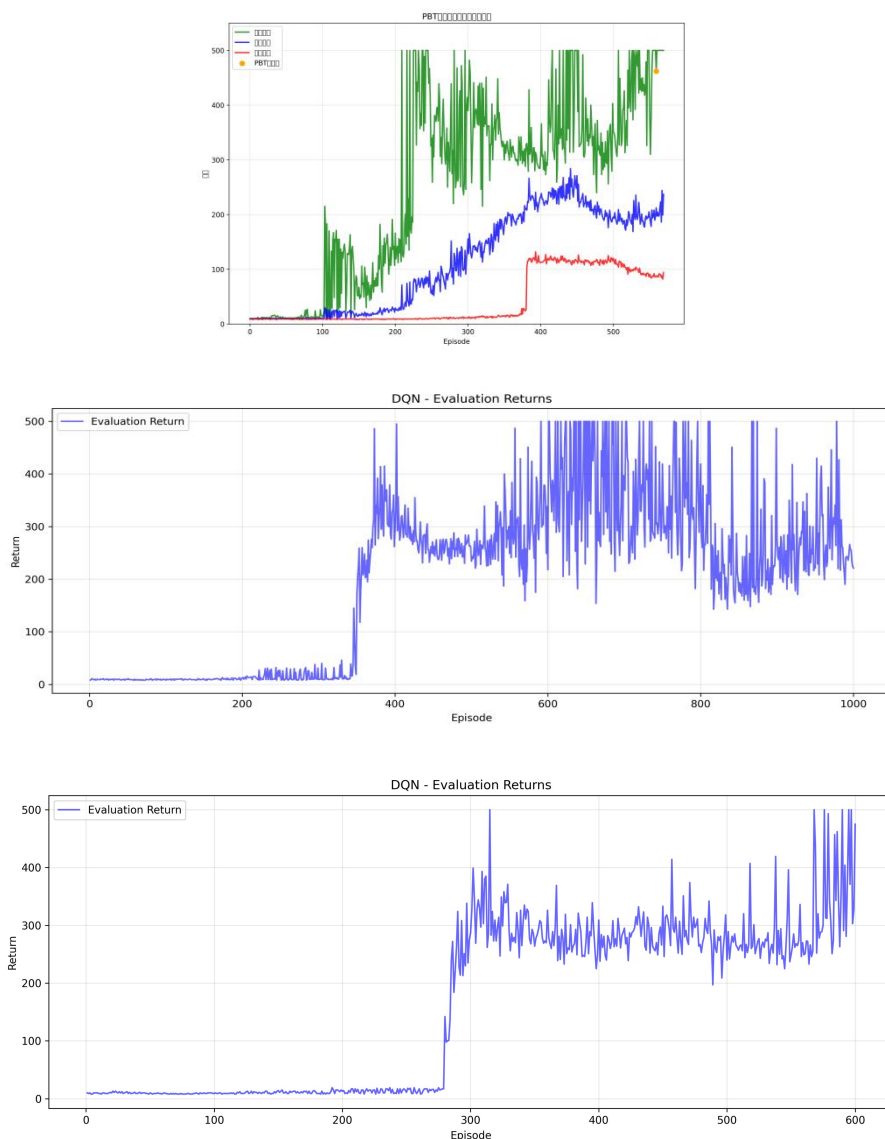
3.4 改进前后模型对比：

1、贝叶斯优化：



左图为注重回报值高的贝叶斯优化结果，右图为注重回报值高且稳定的贝叶斯优化结果。可看出左图和前文中的图表相比回报值骤降的情况减少，右图在回报值高的同时波动情况更少，贝叶斯优化有一定的效果。

2、PBT 训练优化：



最上图为训练和优化过程中的回报值变化趋势，其中我使用了十个智能体并行训练了 600 episodes，绿色线为十个智能体中的最大回报，蓝色线为平均回报，红色线为最小回报。最大值显著大于平均值和最小值，这表明 PBT 算法持续地尝试并发现了一些更具探索性的、能获得高回报的超参数组合，探索力度很大，尽管这些组合的稳定性可能较差。

中图为得到最佳参数组合之后全程使用该参数组合训练 1000 episodes 的 return 值变化趋势，观察发现前 600 episodes 中 return 值较为稳定。

最下图为同参数组合训练 600 episodes 的 return 值变化趋势，已经几乎不存在大幅度骤升骤降的情况。

由此可见 PBT 调参提升效果更加显著。

4 ddqn 与 dqn 对比

4.1 dqn (Deep Q-Network) 介绍:

Deep Q-Network, 即“深度 Q 网络”, 是将深度学习与 Q 学习相结合的强化学习算法, 主要用于解决传统 Q-learning 高维状态空间的局限性问题。

Q-Learning: 传统的 Q 学习算法, 使用表格存储 Q 值。

Q: 代表 Q 函数 (Q-function), 即动作价值函数, 表示在给定状态下执行某个动作的长期期望回报。

使用神经网络来近似 Q 函数, 通过监督学习的方式训练 Q 网络。

4.2 ddqn (Double DQN) 介绍:

DDQN 是为了解决 DQN 的过估计问题而提出的改进算法。

将动作选择和动作评估解耦, 使用 local 网络选择动作, 使用 target 网络评估该动作的 Q 值。

4.3 二者区别:

在框架代码中, DDQN 类继承了 DQN 类, 只重写了 learn 方法。而 learn 方法中, 以下两行不一样:

DQN:

```
Q_targets_next = self.qnetwork_target(next_states).detach().max(1)[0].unsqueeze(1)
Q_targets = rewards + (gamma * Q_targets_next * (1 - dones))
```

DQN 的目标值计算: 目标 Q 值 = 当前奖励 + $\gamma \times$ target 网络的最大 Q 值。

首先用 target 网络计算下一个状态的所有动作 Q 值, 然后直接取最大值用于计算目标值。

(并断开梯度计算, 防止目标值影响 target 网络参数)

即选择动作和评估动作值都使用 target 网络。

DDQN:

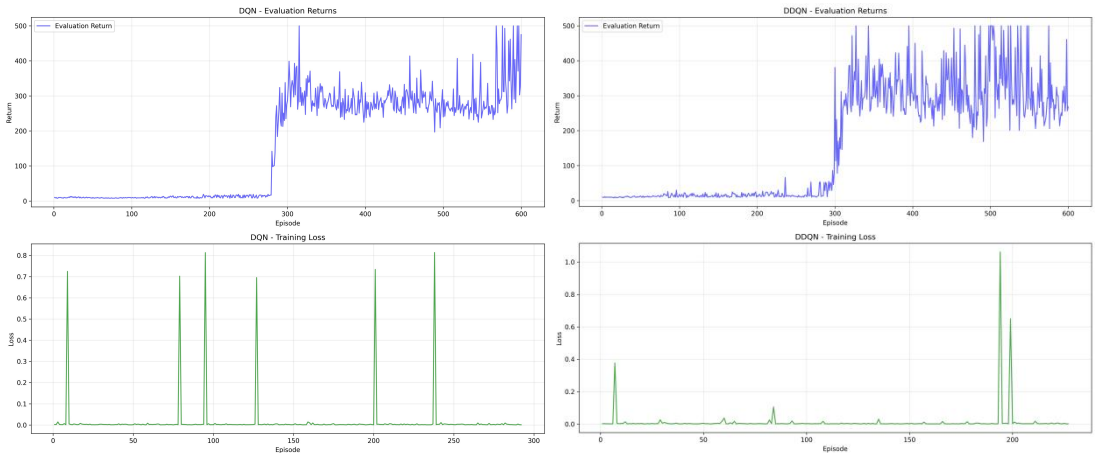
```
action_max = torch.argmax(self.qnetwork_local(next_states), dim=1).unsqueeze(1)
Q_targets = rewards + (gamma * torch.gather(self.qnetwork_target(next_states).detach(), 1,
action_max) * (1 - dones))
```

DDQN 的目标值计算: 目标 Q 值 = 当前奖励 + $\gamma \times$ target 网络对 local 网络选择动作的 Q 值。

首先用 local 网络选择最优动作, 用 target 网络评估刚才选择的动作的 Q 值。

在 DQN 中, 由于使用同一个网络既选择动作又评估价值, 会产生系统性高估。而 DDQN 解耦了动作选择和价值评估, 即动作选择和价值评估由不同网络完成, 两个网络的误差相互独立, 降低了同时高估的概率。

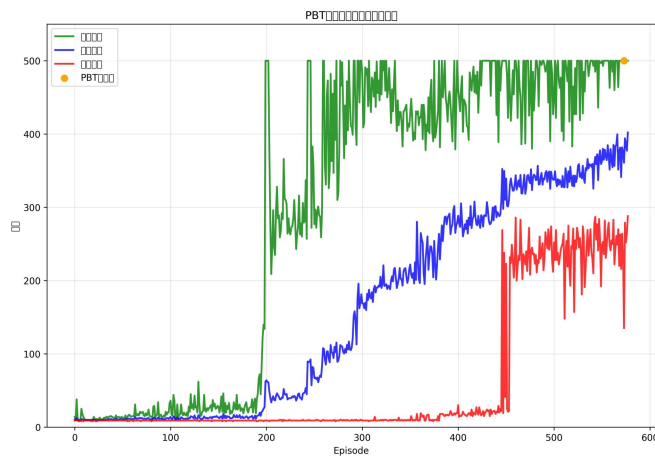
4.4 实验结果的区别：



左图为 DQN 的 return 值和 loss 值的变化曲线，右图为 DDQN 的 return 值和 loss 值的变化曲线，两图为同一种参数组合（上文 PBT 得到的最佳参数组合）下的运行结果。

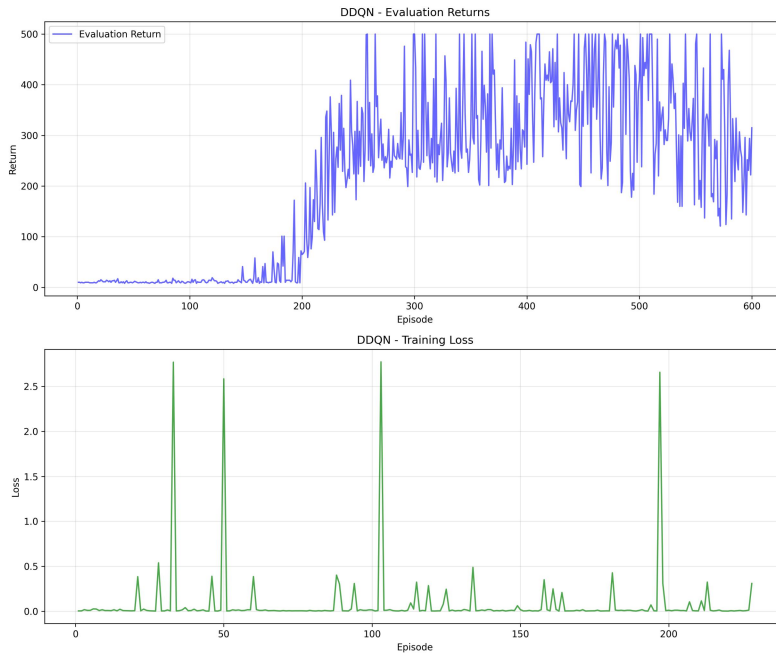
可看出 300 episodes 之后，DDQN 的平均值要略高于 DQN，但是稳定性不如 DQN。我猜测可能是适用于 DQN 的参数配置不适用于 DDQN。

于是对 DDQN 运行 PBT 参数优化：



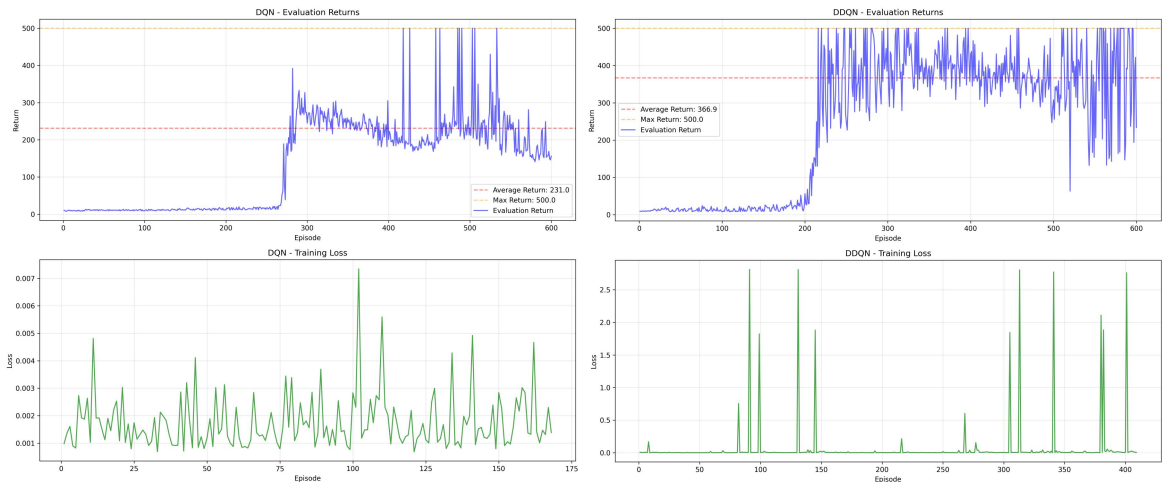
与前文中 DQN 的训练图像对比，已经可看出 DDQN 的 PBT 训练效果优于 DQN。调参之后再次运行 DDQN 算法：

（结果接下页）



可看出 DDQN 算法的 return 值提升在 200 episode 左右, 后续收敛, 较 DQN 的 300 episodes 更快。

为了对比二者 return 值的不同, 我在图像中加入了提升后的平均值的显示。(DQN 取 300 episodes / DDQN 取 200 episodes 后的数据平均值) 现在可以更直观的看出 DDQN 的优势了:



DDQN 在收敛阶段的平均 return 值明显大于 DQN, 说明 DDQN 极大程度的改善了 DQN 的过估计问题, 是对 DQN 算法的良好的改进和提升。

5 reference

[1] [强化学习中关键超参数的详细说明](#) [强化学习参数-CSDN 博客](#)

(https://blog.csdn.net/2401_82947696/article/details/147619655)

[2] [如何调整 DQN 中的超参数 - 知乎](https://zhuanlan.zhihu.com/p/63368522) (<https://zhuanlan.zhihu.com/p/63368522>)

[3] [强化学习中的超参数调整 | 进阶强化学习](#)

(<https://apxml.com/zh/courses/advanced-reinforcement-learning/chapter-8-rl-implementation-optimization/rl-hyperparameter-tuning>)

[4] [上海交大开源 MALib 多智能体并行训练框架, 支持大规模基于种群的多智能体强化学习训练-阿里云开发者社区](#) (<https://developer.aliyun.com/article/822973>)